INSTITUTO TECNOLÓGICO SUPERIOR DE LA
REGIÓN DE LOS LLANOS

INGENIERÍA MECATRÓNICA

Programación Avanzada

Actividad: U2A3. Programa

Perceptrón Multicapa para Seguidor de Línea

Alumnos:

Rubén Álvarez Durán

Juan David Castruita Castañeda

Docente: Osbaldo Aragón Banderas

Fecha: 2026-03-06

TECNM
TECNOLÓGICO NACIONAL DE
MÉXICO

Objetivo de la actividad

Implementar un perceptrón multicapa (MLP) que controle un robot diferencial TURTLE en Proteus, usando 3 sensores de línea como entradas y 2 salidas para controlar los motores, validando su funcionamiento mediante simulación y evidencia multimedia.

Desarrollo

Primeramente, se preparó una simulación en el entorno de Proteus Design Suite, empleando un robot seguidor de línea proporcionado de forma default, conocido como robot diferencial TURTLE. Este se configura para utilizar solo tres sensores de línea (derecho, central e izquierdo). Adicionalmente, emplea dos motores diferenciales, derecho e izquierdo. Este robot es operado mediante el microcontrolador Arduino Uno.

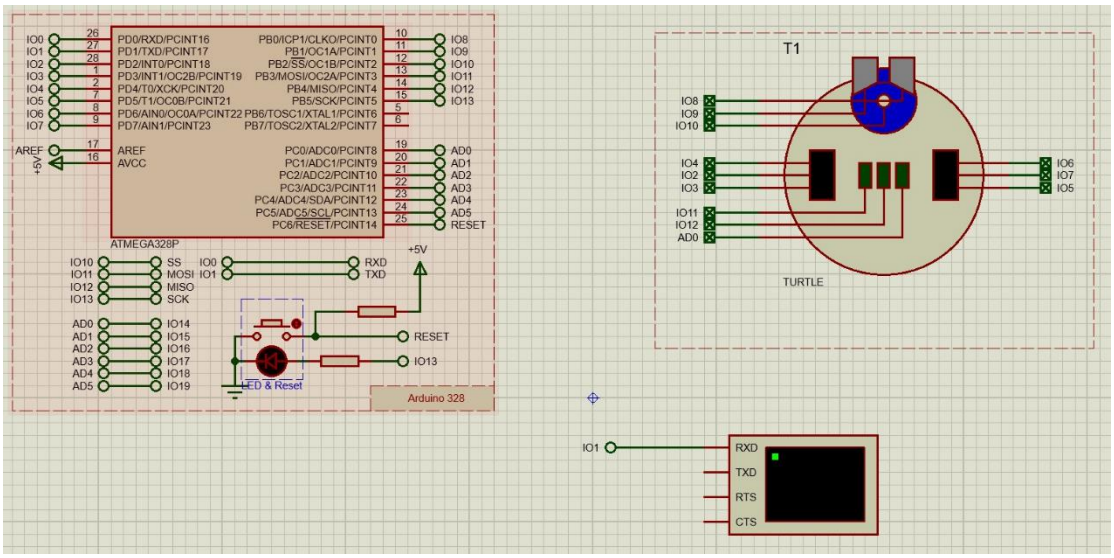


Figura 1. Robot TURTLE simulado en Proteus.

Luego, se define el modelo de perceptrón multicapa, formando lo que se puede describir como una pequeña red neuronal en Python, utilizando un notebook IPYNB. Se diseña un sistema conformado por:

- **Capa de entrada**: 3 neuronas (una por sensor).

- **Capas ocultas**: al menos 1 capa oculta.
- **Capa de salida**: 2 neuronas (una por motor).

De esta forma, se puede realizar el entrenamiento correspondiente del sistema y obtener los valores de los pesos (W) y bias (b), los cuales determinan el funcionamiento del seguidor simulado mediante el control de PWM de los dos motores. Los valores empleados están normalizados entre 0 y 1, determinando la velocidad a la que debe ir cada motor. La función de activación elegida es una de **sigmoide**, la cual convierte los valores en un rango de 0 a 1. W1 conecta la entrada con la capa oculta, mientras que W2 conecta la capa oculta con la salida. Luego de calcular gradientes (indican como ajustar los pesos para reducir error), los pesos se actualizan mediante la taza de aprendizaje, 0.6.

La forma de codificación de entradas es de tipo digital, es decir, las entradas se analizan como un 0 si no se detecta línea, y 1 si se detecta. Con esta información se puede realizar una interpretación de salidas, en donde se determina que el seguidor debe de mantenerse sobre la línea de la pista hecha previamente en Paint. Aquí, se implementa solo la propagación hacia adelante (forward pass) en Arduino. Los **pesos y bias** deben estar cargados explícitamente en el programa (arreglos/constantes) para que el modelo funcione dentro del microcontrolador. El entrenamiento de la red dura 20,000 epochs, cada iteración realizando una predicción de la red.

Finalmente, el programa acomoda los pesos entrenados a arreglos de C, fundamental para la implementación en Arduino. Esta información se exporta en archivos de tipo h y txt.

El microcontrolador Arduino utilizará esta información para primero leer los sensores, y luego calcular la red neuronal para convertir la salida a PWM. El **código Arduino** empleado es el siguiente:

```
#include <Arduino.h>

#include <math.h>
```

```
// ========================
// 1) MAPEO (TURTLE Proteus)
// ========================
#define S_L 11
#define S_C 12
#define S_R A0
#define L_IN1 4
#define L_IN2 2
#define L_EN  3   // PWM
#define R_IN1 6
#define R_IN2 7
#define R_EN  5   // PWM
#define INVERT_SENSORS 0
#define INV_LEFT_DIR   1
#define INV_RIGHT_DIR  0
// ========================
// 2) AJUSTES DE CONTROL
// ========================
const int LOOP_MS = 20;
// Simulación ideal: sin fricción
const int PWM_MAX = 160;
const int PWM_MIN = 40;        // <- clave en simulación ideal

// Filtrado (más rápido para no “llegar tarde” en curvas)
```

```
const float FILTER_ALPHA = 0.60f;

// Telemetría

const int PRINT_EVERY = 5;

// --- Shaping de curvas (esto es lo que arregla que se salga) ---

// Escala SOLO el promedio (velocidad) y BOOSTEA el diferencial.

const float MEAN_GAIN_STRAIGHT = 1.00f;

const float MEAN_GAIN_SOFT     = 0.80f; // 011 / 110

const float MEAN_GAIN_HARD     = 0.60f; // 001 / 100

const float MEAN_GAIN_AMBIG    = 0.75f; // 101 / 111

const float DIFF_BOOST_STRAIGHT = 1.00f;

const float DIFF_BOOST_SOFT     = 1.60f;

const float DIFF_BOOST_HARD     = 2.60f;

const float DIFF_BOOST_AMBIG    = 1.30f;

// En curva dura, fuerza la rueda interna a bajar mucho

const float INNER_CLAMP_HARD = 0.18f;  // 18% de PWM_MAX

// ========================

// 3) PESOS Y BIAS DEL MLP

// Arquitectura: 3 -> 4 -> 2

// Convención: W1[3][4], W2[4][2]

// ========================

const float W1[3][4] = {

  {-1.88253415f,  1.68771088f,  4.11760426f,  1.86243320f},

  {-0.56759626f, -2.38037062f,  3.71695733f, -2.91830683f},

  { 2.19024444f,  1.77338624f,  0.82553440f, -2.09700727f},
```

```
};
const  float  b1[4]  =  {  0.11005893f,    1.10109711f,  -0.74756557f,  -
1.25737357f};
const float W2[4][2] = {
  { 2.88934469f, -2.74164128f},
  {-2.09860039f, -1.83125734f},
  {-1.61372387f,  3.86725640f},
  {-2.76645470f,  2.20006585f},
};
const float b2[2] = { 2.64302588f, -0.53238839f};
// =========================
// 4) ESTADO
// =========================
int last_dir = 0;          // -1 izquierda, +1 derecha
float pwmL_filt = 0.0f;
float pwmR_filt = 0.0f;
int print_div = 0;
// =========================
// 5) AUX
// =========================
static inline float sigmoidf(float z) {
  if (z > 60.0f) z = 60.0f;
  if (z < -60.0f) z = -60.0f;
  return 1.0f / (1.0f + expf(-z));
```

```
}
static inline float clamp01(float v) {
  if (v < 0.0f) return 0.0f;
  if (v > 1.0f) return 1.0f;
  return v;
}
static inline int normToPwm(float u) {
  u = clamp01(u);
  float pwm = PWM_MIN + u * (PWM_MAX - PWM_MIN);
  if (pwm < 0.0f) pwm = 0.0f;
  if (pwm > PWM_MAX) pwm = PWM_MAX;
  return (int)(pwm + 0.5f);
}
static inline void setLeftMotorForward(int pwm) {
  pwm = constrain(pwm, 0, 255);
  if  (!INV_LEFT_DIR)  {  digitalWrite(L_IN1,  HIGH);  digitalWrite(L_IN2,
LOW); }
  else            { digitalWrite(L_IN1, LOW);  digitalWrite(L_IN2, HIGH);
}
  analogWrite(L_EN, pwm);
}


static inline void setRightMotorForward(int pwm) {
  pwm = constrain(pwm, 0, 255);
```

```
  if (!INV_RIGHT_DIR) { digitalWrite(R_IN1, HIGH); digitalWrite(R_IN2,
LOW); }
  else                  { digitalWrite(R_IN1, LOW);  digitalWrite(R_IN2,
HIGH); }
  analogWrite(R_EN, pwm);
}
static inline void stopMotors() {
  analogWrite(L_EN, 0);
  analogWrite(R_EN, 0);
}
static inline int readSensorDigital(int pin) {
  int s = digitalRead(pin);
#if INVERT_SENSORS
  s = !s;
#endif
  return s ? 1 : 0;
}
// ========================
// 6) MLP FORWARD (consistente con W1[3][4], W2[4][2])
// Oculta: SIGMOIDE (si tu entrenamiento fue sigmoide)
// ========================
static inline void mlpForward(const float x[3], float y[2]) {
  float h[4];
```

```
  for (int j = 0; j < 4; j++) {
    float z = b1[j];
    z += x[0] * W1[0][j];
    z += x[1] * W1[1][j];
    z += x[2] * W1[2][j];
    h[j] = sigmoidf(z);
  }
  for (int k = 0; k < 2; k++) {
    float z = b2[k];
    z += h[0] * W2[0][k];
    z += h[1] * W2[1][k];
    z += h[2] * W2[2][k];
    z += h[3] * W2[3][k];
    y[k] = sigmoidf(z); // 0..1
  }
}
// ========================
// 7) SETUP
// ========================
void setup() {
  pinMode(S_L, INPUT);
  pinMode(S_C, INPUT);
  pinMode(S_R, INPUT);
  pinMode(L_IN1, OUTPUT);
```

```
  pinMode(L_IN2, OUTPUT);
  pinMode(L_EN,  OUTPUT);
  pinMode(R_IN1, OUTPUT);
  pinMode(R_IN2, OUTPUT);
  pinMode(R_EN,  OUTPUT);
  stopMotors();
  Serial.begin(115200);
  Serial.println("t_ms,pat,sL,sC,sR,yL,yR,cmdL,cmdR,pwmL,pwmR,last_dir");
}
// =========================
// 8) LOOP
// =========================
void loop() {
  // 8.1 Sensores
  int sL = readSensorDigital(S_L);
  int sC = readSensorDigital(S_C);
  int sR = readSensorDigital(S_R);
  int pattern = (sL << 2) | (sC << 1) | sR;
  // 8.2 Memoria de último lado
  if (sL && !sR) last_dir = -1;
  else if (sR && !sL) last_dir = +1;

  // 8.3 MLP
  float x[3] = {(float)sL, (float)sC, (float)sR};
```

```
  float y[2];
  mlpForward(x, y);
  int cmdL = normToPwm(y[0]);
  int cmdR = normToPwm(y[1]);
  // 8.4 Shaping: escalar promedio (velocidad) y boostear diferencial
(giro)
  float mean = 0.5f * (cmdL + cmdR);
  float diff = 0.5f * (cmdL - cmdR);
  float meanGain = MEAN_GAIN_STRAIGHT;
  float diffBoost = DIFF_BOOST_STRAIGHT;
  if (pattern == 0b011 || pattern == 0b110) {
    meanGain  = MEAN_GAIN_SOFT;
    diffBoost = DIFF_BOOST_SOFT;
  } else if (pattern == 0b001 || pattern == 0b100) {
    meanGain  = MEAN_GAIN_HARD;
    diffBoost = DIFF_BOOST_HARD;
  } else if (pattern == 0b101 || pattern == 0b111) {
    meanGain  = MEAN_GAIN_AMBIG;
    diffBoost = DIFF_BOOST_AMBIG;
  }

  float base = mean * meanGain;
  float d    = diff * diffBoost;
  float targetL = base + d;
```

```
  float targetR = base - d;

  // 8.5 Clamp rueda interna en curvas duras (evita que “no gire suficiente”)

  if (pattern == 0b001 || pattern == 0b011) { // línea a la derecha => girar derecha => rueda derecha interna

    float clampInner = INNER_CLAMP_HARD * PWM_MAX;

    if (targetR > clampInner) targetR = clampInner;

  } else if (pattern == 0b100 || pattern == 0b110) { // línea a la izquierda => rueda izquierda interna

    float clampInner = INNER_CLAMP_HARD * PWM_MAX;

    if (targetL > clampInner) targetL = clampInner;

  }

  // 8.6 Supervisor línea perdida (000)

  if (pattern == 0b000) {

    // Búsqueda fuerte

    if (last_dir < 0) {

      targetL = 0.25f * PWM_MAX;

      targetR = 0.75f * PWM_MAX;

    } else if (last_dir > 0) {

      targetL = 0.75f * PWM_MAX;

      targetR = 0.25f * PWM_MAX;

    } else {

      targetL = 0.55f * PWM_MAX;

      targetR = 0.55f * PWM_MAX;

    }
```

```
}
// 8.7 Saturación final
if (targetL < 0) targetL = 0;
if (targetR < 0) targetR = 0;
if (targetL > PWM_MAX) targetL = PWM_MAX;
if (targetR > PWM_MAX) targetR = PWM_MAX;
// 8.8 Filtrado
pwmL_filt += FILTER_ALPHA * (targetL - pwmL_filt);
pwmR_filt += FILTER_ALPHA * (targetR - pwmR_filt);
int pwmL = (int)(pwmL_filt + 0.5f);
int pwmR = (int)(pwmR_filt + 0.5f);
// 8.9 Aplicar
setLeftMotorForward(pwmL);
setRightMotorForward(pwmR);
// 8.10 Telemetría
print_div++;
if (print_div >= PRINT_EVERY) {
  print_div = 0;
  Serial.print(millis()); Serial.print(",");
  Serial.print(pattern);  Serial.print(",");
  Serial.print(sL);       Serial.print(",");
  Serial.print(sC);       Serial.print(",");
  Serial.print(sR);       Serial.print(",");
  Serial.print(y[0], 4);  Serial.print(",");
```

```
    Serial.print(y[1], 4);  Serial.print(",");
    Serial.print(cmdL);     Serial.print(",");
    Serial.print(cmdR);     Serial.print(",");
    Serial.print(pwmL);     Serial.print(",");
    Serial.print(pwmR);     Serial.print(",");
    Serial.println(last_dir);
  }
  delay(LOOP_MS);
}
```

En este código, se pueden apreciar las constantes de pesos y bias finales:

```
const float W1[3][4] = {
  {-1.88253415f,  1.68771088f,  4.11760426f,  1.86243320f},
  {-0.56759626f, -2.38037062f,  3.71695733f, -2.91830683f},
  { 2.19024444f,  1.77338624f,  0.82553440f, -2.09700727f},
};const float W2[4][2] = {
  { 2.88934469f, -2.74164128f},
  {-2.09860039f, -1.83125734f},
  {-1.61372387f,  3.86725640f},
  {-2.76645470f,  2.20006585f},
};
const float b2[2] = { 2.64302588f, -0.53238839f};
// ========================
// 4) ESTADO
// ========================
```

```
int last_dir = 0;          // -1 izquierda, +1 derecha

float pwmL_filt = 0.0f;

float pwmR_filt = 0.0f;

int print_div = 0;
```

La arquitectura final del sistema de entrenamiento queda así:

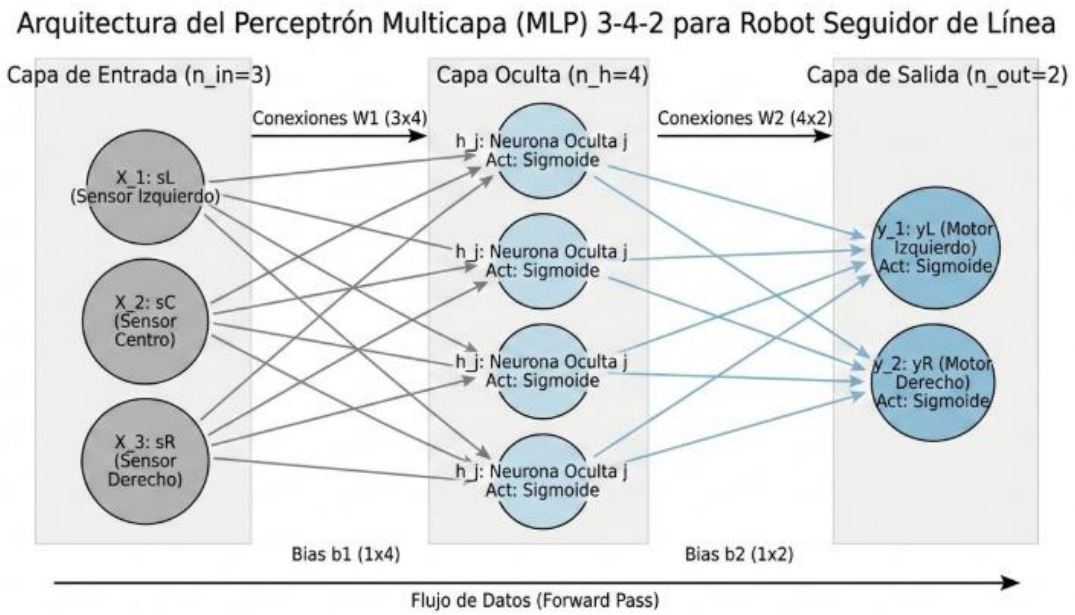


Figura 2. Arquitectura del MLP.

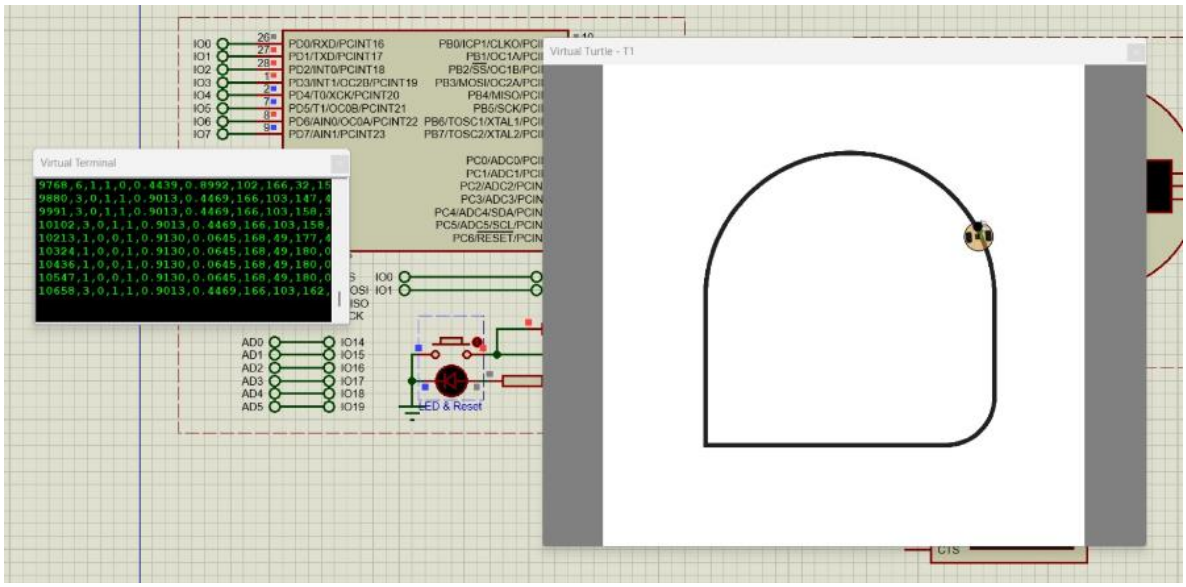
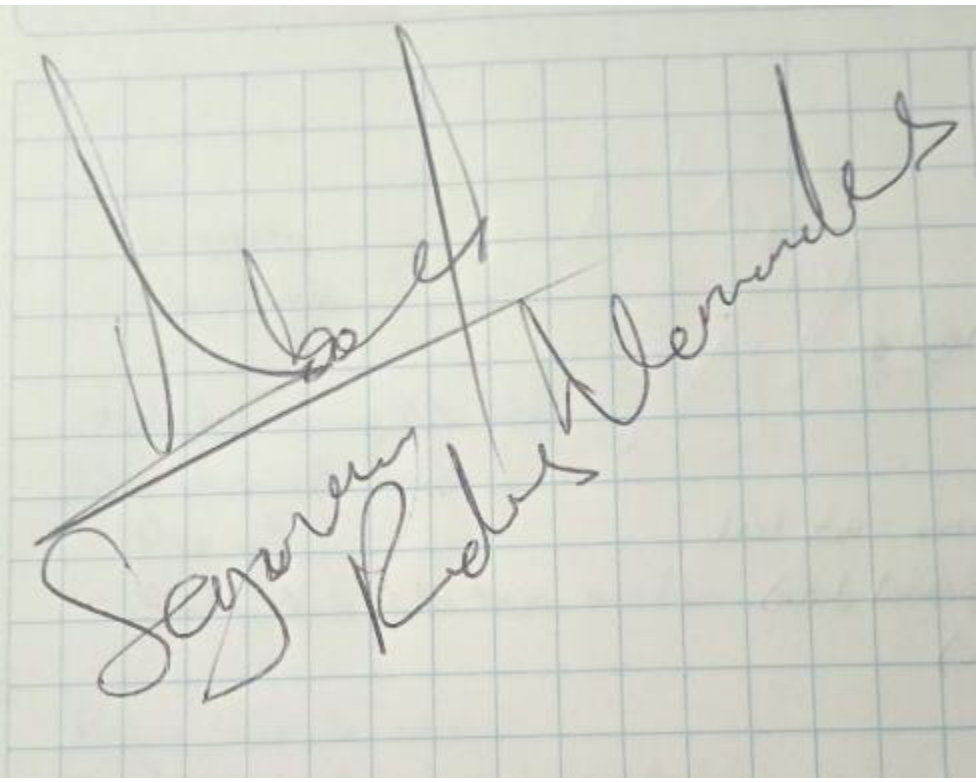


Figura 3. Seguidor de línea funcionando en la simulación.

Al probar la simulación en Proteus, se demuestra que el robot sigue la línea de forma estable en condiciones normales y se recupera cuando se desplaza ligeramente fuera de la línea.

Enlace a **video de simulación**: https://youtu.be/kTHh8CWH9Ic?si=vrwovfkoT_Sc6T5v

Firma de revisado